

Document Intelligence Platform

Intelligent Document Extraction, Validation & API Platform

AI Engineer Internship — Technical Case Study Submission

FastAPI · PyMuPDF · Tesseract OCR · Rule-based NLP Extraction · SQLite · Docker

Problem & Objective

The challenge

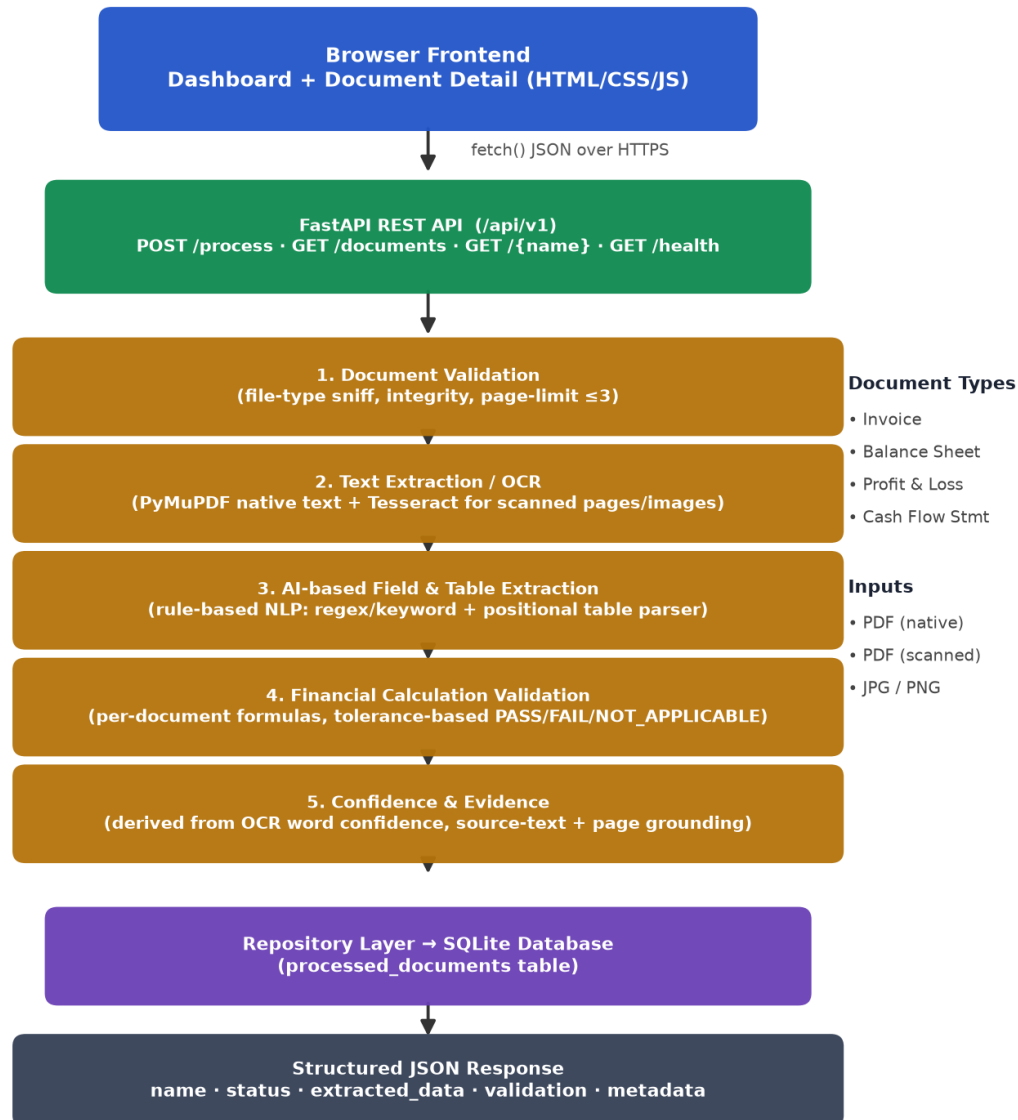
- Financial teams receive invoices & statements as native or scanned PDF/JPG/PNG.
- Varying layouts, OCR noise and inconsistent line items make manual extraction slow.
- Need: a reusable service that extracts everything, validates the numbers, and
- makes results available via a dashboard + REST API.

What this submission delivers

- Full pipeline: validate → OCR/text extract → AI-based field & table extract →
- financial validation → confidence/evidence → persist → dashboard + REST API.
- All 4 document types: Invoice, Balance Sheet, Profit & Loss, Cash Flow Statement.
- Deployed, documented, tested — Dockerized for one-command deployment.

Architecture

Document Intelligence Platform — Architecture



AI-Based Extraction: Why Rule-Based, Not an LLM

- Zero cost, zero external dependency — no API key, no rate limit risk during evaluation,
- runs fully offline.
- Deterministic & explainable — same input always gives the same output.
- Confidence scores are derived from the OCR engine's real per-word confidence,
- never an invented number.
- The 3 financial statement types share one consistent tabular layout — a positional
- label/value table parser handles this very reliably.

Trade-off, stated honestly: a hosted LLM would generalize better to unseen layouts — see "Known Limitations" in the README.

Extraction Pipeline

services/ package

1. Document Validation

Magic-byte file-type sniffing (not trusted extension), integrity check, 3-page limit.

2. OCR Service

PyMuPDF native text extraction; scanned pages/images OCR'd via Tesseract with word-level confidence.

3. Financial Statement Parser

Shared positional table parser: groups words into rows, separates schedule refs from period-value columns, tracks section headers for reconciliation.

4. Extraction Service

Per document_type: regex/keyword extraction (invoices) or keyword-mapped table lookups (statements) + full line-item arrays.

5. Financial Validation Service

Spec formulas §4.4 per document type & period; tolerance-based
PASS / FAIL / NOT_APPLICABLE.

Financial Validation — Examples

Balance Sheet

Total Capital & Liabilities $\approx$ Total Assets, independently per reporting period,
plus component-sum reconciliation for each side.

Profit & Loss

Interest Earned + Other Income $\approx$ Total Income · Total Income – Total Expenditure $\approx$
Net Profit before Minority Interest · Current + Brought Forward $\approx$ Total for Appropriation

Cash Flow Statement

Operating + Investing + Financing + FX $\approx$ Net Increase in Cash · Opening + Net Increase
 $\approx$ Closing Cash. Parentheses treated as negative.

*Tolerance: $\max(\pm 1.0 \text{ absolute}, \pm 1\% \text{ relative})$ — a check is NOT_APPLICABLE, never FAIL,
when a required field wasn't found — nothing is ever assumed or invented.*

Demonstrated Scenarios

see `sample_outputs/`

PASS

`balance_sheet_pass.json` / `cash_flow_statement_pass.json` / `invoice_pass.json`

VALIDATION FAIL

`profit_and_loss_validation_fail.json` — a real OCR digit-misread on "Total Expenditure" is correctly caught by the reconciliation check, left un-fixed on purpose.

MISSING FIELDS

`invoice_low_quality_missing_fields.json` — heavily stamped receipt; unreadable fields report null (and low confidence) rather than a guess.

UNSUPPORTED FILE

`unsupported_file_failed.json` — .txt upload rejected gracefully, no crash.

Technology Stack & Repository Structure

- FastAPI + Pydantic v2 — API, validation, auto Swagger docs at /docs
- PyMuPDF — native PDF text + rasterizing scanned pages/images
- Tesseract (pytesseract) — free, local, offline OCR with per-word confidence
- SQLAlchemy + SQLite — zero-config persistence, swappable to Postgres via DATABASE_URL
- Plain HTML/CSS/JS frontend (Jinja2 shells) — no Node/React build step
- pytest — 27 automated tests (file validation, financial validation, extraction, API)
- Docker — one Dockerfile deploys the whole app to Render / Railway / Koyeb

Known Limitations & Production Roadmap

Known Limitations

- Rule-based extraction is tuned to the provided dataset's layouts; very different
- templates need their keyword lists extended.
- OCR occasionally misreads a digit on low-resolution/noisy scans (evidence text lets
- a reviewer verify every value quickly).
- SQLite on a free-tier deploy resets on redeploy unless a persistent disk is attached.

For Production

- Managed Postgres + persistent file storage for originals.
- Optional LLM fallback for low-confidence documents, on top of the free default path.
- Background job queue for processing, with auth + rate limiting on the API.